

**EXERCISE 3.3**

Using the same idea, implement the function `setHead` for replacing the first element of a `List` with a different value.

3.3.1 The efficiency of data sharing

Data sharing often lets us implement operations more efficiently. Let's look at a few examples.

**EXERCISE 3.4**

Generalize `tail` to the function `drop`, which removes the first `n` elements from a list. Note that this function takes time proportional only to the number of elements being dropped—we don't need to make a copy of the entire `List`.

```
def drop[A](l: List[A], n: Int): List[A]
```

**EXERCISE 3.5**

Implement `dropWhile`, which removes elements from the `List` prefix as long as they match a predicate.

```
def dropWhile[A](l: List[A], f: A => Boolean): List[A]
```

A more surprising example of data sharing is this function that adds all the elements of one list to the end of another:

```
def append[A](a1: List[A], a2: List[A]): List[A] =  
  a1 match {  
    case Nil => a2  
    case Cons(h, t) => Cons(h, append(t, a2))  
  }
```

Note that this definition only copies values until the first list is exhausted, so its runtime and memory usage are determined only by the length of `a1`. The remaining list then just points to `a2`. If we were to implement this same function for two arrays, we'd be forced to copy all the elements in both arrays into the result. In this case, the immutable linked list is much more efficient than an array!